МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ • КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ И ПРОГРАММИРОВАНИЯ

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Тема: «Деревья. Парсинг скобочной записи и сбалансированное АВЛ-дерево»

Дисциплина: Алгоритмы и структуры данных
Преподаватель: Молдовян Дмитрий Николаевич
Выполнил: ст. гр. 5372 Павленко П. С.

1. ПОСТАНОВКА ЗАДАЧИ И ЦЕЛЬ РАБОТЫ

Задание к лабораторной работе №3 («Деревья»):

Реализовать: 1) Обычное двоичное дерево (не поиска и не балансирующее) с реализацией обхода этого дерева в глубину через рекурсию. 2) Алгоритм для парсинга скобочной записи дерева. Пример: `(8 (9 (5)) (1))` — корень 8, левый сын 9, правый 1, левый сын узла 9 — 5. 3) **АВЛ-дерево** с реализацией алгоритмов вставки, удаления, поиска, обхода в ширину и обхода в глубину (все 3 вида, использовать **итеративный вариант**).
Продемонстрировать: Чтение скобочной записи из файла; построение двоичного дерева или сообщение об ошибке (неверные символы, скобки, не двоичное дерево и т.п.); обход полученного дерева и создание АВЛ-дерева; вывод всех узлов 4 способами (в ширину и 3 в глубину).

2. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ И АРХИТЕКТУРА СТРУКТУР

Иерархические древовидные структуры применяются для представления вложенных данных и быстрого логарифмического поиска. В работе реализованы две взаимосвязанные модели деревьев:

- **Произвольное двоичное дерево и скобочный парсер:** каждый узел имеет не более двух потомков (`left` и `right`) без ограничений на порядок ключей. Грамматика скобочной записи имеет вид `(Корень (ЛевоеПоддерево) (ПравоеПоддерево))`. При синтаксическом анализе проверяется баланс круглых скобок, отсутствие посторонних символов и условие ветвления (степень исхода каждого узла ≤ 2; при обнаружении 3-го сына фиксируется ошибка *«не двоичное дерево»*).
- **Сбалансированное АВЛ-дерево (AVL Tree):** двоичное дерево поиска (для любого узла ключи в левом поддереве меньше, а в правом — больше ключа узла), в котором для каждой вершины поддерживается инвариант сбалансированности: модуль разности высот правого и левого поддеревьев не превышает единицы: $|bf(v)| = |h(R) - h(L)| \leq 1$. При нарушении баланса ($bf = \pm 2$) после вставки или удаления выполняются малые (LL, RR) или большие (LR, RL) балансирующие повороты за **O(1)**, гарантирующие высоту дерева и сложность операций **O(log n)**.

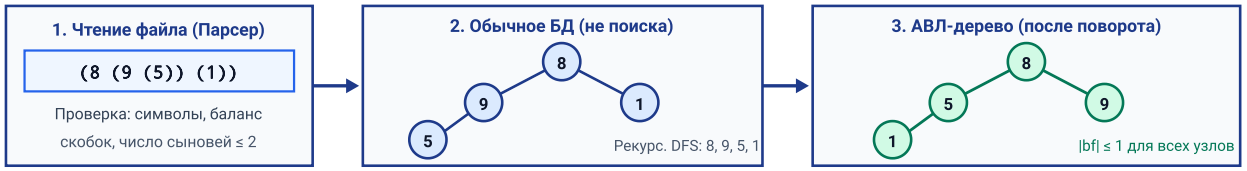


Рис. 1. Схема построения обычного двоичного дерева из строки (8 (9 (5)) (1)) и его перестройки в АВЛ-дерево

3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ: ПАРСЕР СКОБОЧНОЙ ЗАПИСИ И АВЛ-ДЕРЕВО (C++)

```
#include <iostream> #include <fstream> #include <string> #include <vector> #include <stack> #include <queue> #include <stdexcept> using namespace std; // 1. Узел обычного двоичного дерева и рекурсивный парсер скобочной записи struct BTNode { int val; BTNode *left = nullptr, *right = nullptr; BTNode(int v) : val(v) {} }; void skipSpaces(const string& s, size_t& pos) { while (pos < s.size() && isspace((unsigned char)s[pos])) pos++; } BTNode* parseSubtree(const string& s, size_t& pos) { skipSpaces(s, pos); if (pos >= s.size() || s[pos] != '(') throw runtime_error("Ошибка: ожидалась открывающая скобка '('"); pos++; // Пропускаем '(' skipSpaces(s, pos); if (pos < s.size() && s[pos] == ')') { pos++; return nullptr; } // Пустое поддерево () int sign = 1; if (pos < s.size() && s[pos] == '-') { sign = -1; pos++; } if (pos >= s.size() || !isdigit((unsigned char)s[pos])) throw runtime_error("Ошибка: недопустимый символ или отсутствует значение узла"); int val = 0; while (pos < s.size() && isdigit((unsigned char)s[pos])) val = val * 10 + (s[pos++] - '0'); BTNode* node = new BTNode(val * sign); int childCount = 0; while (true) { skipSpaces(s, pos); if (pos >= s.size()) throw runtime_error("Ошибка: нарушен баланс скобок (нет ')')"); if (s[pos] == '(') { childCount++; if (childCount > 2) throw runtime_error("Ошибка: у узла более 2 потомков (не двоичное дерево)"); BTNode* child = parseSubtree(s, pos); if (childCount == 1) node->left = child; else node->right = child; } else if (s[pos] == ')') { pos++; break; } else { throw runtime_error("Ошибка: посторонний символ в скобочной записи"); } } return node; } // Рекурсивный обход обычного двоичного дерева в глубину (DFS Pre-order) void traverseBinaryTreeRecursive(BTNode* root, vector<int>& out) { if (!root) return; out.push_back(root->val); traverseBinaryTreeRecursive(root->left, out); traverseBinaryTreeRecursive(root->right, out); } // 2. Узел АВЛ-дерева и функции балансировки (повороты, вставка, удаление, поиск) struct AVLNode { int key, h = 1; AVLNode *left = nullptr, *right = nullptr; AVLNode(int k) : key(k) {} }; int height(AVLNode* n) { return n ? n->h : 0; } int bFactor(AVLNode* n) { return n ? height(n->right) - height(n->left) : 0; } void fixHeight(AVLNode* n) { n->h = max(height(n->left), height(n->right)) + 1; } AVLNode* rotateRight(AVLNode* p) { AVLNode* q = p->left; p->left = q->right; q->right = p; fixHeight(p); fixHeight(q); return q; } AVLNode* rotateLeft(AVLNode* q) { AVLNode* p = q->right; q->right = p->left; p->left = q; fixHeight(q); fixHeight(p); return p; } AVLNode* balance(AVLNode* p) { fixHeight(p); if (bFactor(p) == 2) { if (bFactor(p->right) < 0) p->right = rotateRight(p->right); return rotateLeft(p); } if (bFactor(p) == -2) { if (bFactor(p->left) > 0) p->left = rotateLeft(p->left); return rotateRight(p); } return p; }
```

4. СВОДКА АЛГОРИТМОВ ИТЕРАТИВНОГО ОБХОДА АВЛ-ДЕРЕВА

Вид обхода	Порядок посещения	Вспомогательная структура	Применение и результат на АВЛ-дереве
В ширину (BFS)	По уровням сверху вниз	Очередь (`std::queue`)	Поуровневая печать дерева: 8 → 5, 9 → 1
Прямой (NLR)	Корень → Левое → Правое	Стек (`std::stack`)	Копирование и сериализация структуры: 8, 5, 1, 9
Симметричный (LNR)	Левое → Корень → Правое	Стек + текущий указатель	Вывод ключей в строго отсортированном виде: 1, 5, 8, 9
Обратный (LRN)	Левое → Правое → Корень	Стек + указатель `lastVisited`	Обработка поддеревьев перед корнем: 1, 5, 9, 8

5. ВСТАВКА, УДАЛЕНИЕ, ПОИСК И 4 ИТЕРАТИВНЫХ ОБХОДА АВЛ-ДЕРЕВА

```
AVLNode* insertAVL(AVLNode* p, int k) { if (!p) return new AVLNode(k); if (k < p->key) p->left = insertAVL(p->left, k); else if (k > p->key) p->right = insertAVL(p->right, k); return balance(p); } AVLNode* findMin(AVLNode* p) { return p->left ? findMin(p->left) : p; } AVLNode* removeMin(AVLNode* p) { if (!p->left) return p->right; p->left = removeMin(p->left); return balance(p); } AVLNode* removeAVL(AVLNode* p, int k) { if (!p) return nullptr; if (k < p->key) p->left = removeAVL(p->left, k); else if (k > p->key) p->right = removeAVL(p->right, k); else { AVLNode *q = p->left, *r = p->right; delete p; if (!r) return q; AVLNode* minNode = findMin(r); minNode->right = removeMin(r); minNode->left = q; return balance(minNode); } return balance(p); } bool searchAVL(AVLNode* root, int k) { while (root) { if (k == root->key) return true; root = (k < root->key) ? root->left : root->right; } return false; } // Итеративный обход в ширину (1. BFS) и 3 итеративных обхода в глубину (2. PreOrder, 3. InOrder, 4. PostOrder) void printBFS(AVLNode* root) { if (!root) return; queue<AVLNode*> q; q.push(root); while (!q.empty()) { AVLNode* c = q.front(); q.pop(); cout << c->key << " "; if (c->left) q.push(c->left); if (c->right) q.push(c->right); } } void printPreOrderIter(AVLNode* root) { if (!root) return; stack<AVLNode*> st; st.push(root); while (!st.empty()) { AVLNode* c = st.top(); st.pop(); cout << c->key << " "; if (c->right) st.push(c->right); if (c->left) st.push(c->left); } } void printInOrderIter(AVLNode* root) { stack<AVLNode*> st; AVLNode* c = root; while (c || !st.empty()) { while (c) { st.push(c); c = c->left; } c = st.top(); st.pop(); cout << c->key << " "; c = c->right; } } void printPostOrderIter(AVLNode* root) { stack<AVLNode*> st; AVLNode *c = root, *last = nullptr; while (c || !st.empty()) { if (c) { st.push(c); c = c->left; } else { AVLNode* peek = st.top(); if (peek->right && last != peek->right) c = peek->right; else { cout << peek->key << " "; last = peek; st.pop(); } } } }
```

6. ДЕМОНСТРАЦИЯ ЧТЕНИЯ ИЗ ФАЙЛА И РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Тест	Содержимое файла `tree.txt`	Результат Шага 1 (Парсер БД)	Результат Шага 2–3 (АВЛ и вывод 4 способами)
Тест 1 (пример)	`(8 (9 (5)) (1))`	Успешно построено БД. Рекурс. DFS: 8 9 5 1	**BFS:** 8 5 9 1 • **PreOrder:** 8 5 1 9 **InOrder:** 1 5 8 9 • **PostOrder:** 1 5 9 8
Тест 2 (7 узлов)	`(10 (20 (30) (40)) (50 (60) (70)))`	Успешно построено БД. Рекурс. DFS: 10 20 30 40 50 60 70	**BFS:** 40 20 60 10 30 50 70 **InOrder:** 10 20 30 40 50 60 70 (высота 3)
Тест 3	`(8 (9 (5)) (1) (4))`	**Ошибка:** у узла 8 более 2 потомков (не двоичное дерево)	Выполнение остановлено, выведено диагностическое сообщение
Тест 4	`(8 (9 (5) (1))`	**Ошибка:** нарушен баланс скобок (отсутствует `')'`)	Выполнение остановлено, выведено диагностическое сообщение
Тест 5	`(8 (9 (5a)) (1))`	**Ошибка:** посторонний символ `'a'` в скобочной записи	Выполнение остановлено, выведено диагностическое сообщение

7. ВЫВОДЫ ПО РАБОТЕ

В ходе выполнения лабораторной работы № 3 были реализованы структура обычного двоичного дерева с рекурсивным обходом в глубину и синтаксический анализатор чтения скобочной записи из файла с полной диагностикой ошибок (контроль баланса скобок, недопустимых символов и ограничения не более двух сыновей у каждого узла). На основе рекурсивного обхода построенного дерева выполнено формирование самобалансирующегося **АВЛ-дерева** с операциями вставки, удаления и поиска за время **O(log n)**, а также продемонстрирован вывод всех узлов дерева 4 итеративными способами (в ширину через очередь и 3 в глубину через стек).